

Deep Reinforcement Learning for QoS-Aware IoT Service Composition: The PD3QND Approach

Yi Chen and Lianglun Cheng

*School of Computer
Guangdong University of Technology
Guangzhou, Guang Dong Province, China
2112105266@mail2.gdut.edu.cn*

Tao Wang

*School of Automation
Guangdong University of Technology
Guangzhou, Guang Dong Province, China
wangtao_cps@gdut.edu.cn*

Abstract—With the rapid development of the Internet of Things (IoT), many heterogeneous IoT devices provide atomic services that need to be integrated and combined to form complex services, meeting intricate user demands. In IoT scenarios, the dynamic nature of the environment often leads to fluctuations in the quality of service of atomic services or even their unavailability. Common heuristic composition optimization algorithms struggle to adapt to dynamic environments and require intricate design processes and parameter adjustments. This study introduces a deep reinforcement learning-based optimization algorithm, PD3QND, which incorporates fundamental DQN, noise networks, prioritized experience replay, double dueling architecture, and demonstration learning. Experiments show that, compared to heuristic algorithms and methods like DQN, our algorithm can adaptively balance exploitation and exploration when facing dynamic quality of service (QoS) changes in manufacturing IoT environments. It avoids the cold start problem and robustly and efficiently searches the solution space, demonstrating faster convergence speed and more robust adaptability.

Keywords—Internet of Things (IoT); Service Composition; QoS-aware; Deep Reinforcement Learning;

I. INTRODUCTION

The Internet of Things (IoT) development brings new challenges and opportunities. Service-oriented architecture is becoming increasingly important in IoT, abstracting applications into services for flexible combinations. However, orchestrating services that meet quality standards remains a significant challenge in resource-limited IoT and edge computing environments. Service composition is a multi-objective combinatorial optimization problem in the NP-hard category. As the number of services increases, the complexity of the problem also rises. However, existing solutions, including metaheuristic algorithms, often fall short when dealing with dynamic changes in service quality. As a result, researchers are looking into new optimization methods, such as hybrid metaheuristics or machine learning algorithms, and have achieved some results.

This paper begins with a review of research related to service composition, followed by a detailed elaboration on the formalization of the service composition problem. We then

present the detailed process of our service composition approach, PD3QND, based on deep reinforcement learning. Through experiments, the effectiveness of the PD3QND algorithm is validated, and the results are analyzed.

II. RELATIVE WORK

In recent years, the QoS-aware service composition problem has garnered significant attention from the academic community. Traditional heuristic algorithms have demonstrated their effectiveness in specific scenarios. Reference [1] proposed the Dynamic Ant Colony Genetic Hybrid Algorithm (DAAGA) to solve the cloud service composition optimization problem. This method significantly enhances precision and stability when dealing with large-scale cloud service compositions. Reference [2] employed the Multi-objective Discrete Gray Wolf Optimization (MDGWO) method to select the optimal composite Web services, yet their approach falls short in supporting dynamic Web service composition. Reference [3], focusing on the rapid expansion of IoT and massive growth in cloud computing, introduced an enhanced eagle strategy algorithm to reduce computation time for large-scale cloud computing service composition. With the swift advancement of machine learning technologies, new methods rooted in this domain have emerged and been applied to service composition optimization. Reference [4] unveiled a deep supervised learning framework combining Graph Convolutional Networks and attention mechanisms, striving to construct high-QoS composite services in a limited time. Reinforcement learning, a subset of machine learning, learns optimal strategies through environmental interactions. Reference [5] incorporated skyline technology with the Q-learning algorithm, presenting a reinforcement learning service composition model, although their approach did not fully consider the dynamic nature of service QoS. Reference [6] introduced a deep reinforcement learning method (PPDRL) that combines pre-training, maximum likelihood estimation, and policy rating mechanisms designed explicitly for QoS-aware adaptive service composition challenges. Reference [7] delved into the Deep Deterministic Policy Gradient (DDPG) method to address the service composition issue in dynamic cloud manufacturing environments, showcasing outstanding adaptability, robustness, and scalability.

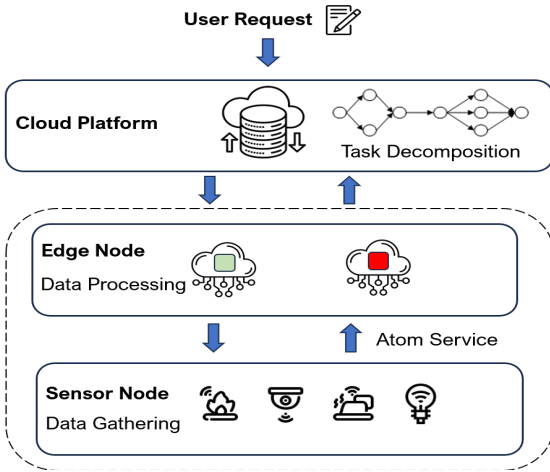


Figure 1. The hierarchical structure of the IIoT environment.

III. SYSTEM MODEL AND PROBLEM DESCRIPTION

A. Service Composition Problem Description

The problem is based on a typical Industrial Internet of Things environment, comprising three main layers: the Cloud platform, Edge nodes, and IoT nodes. Fig. 1 illustrates the hierarchical structure of the IIoT environment. In this model, IoT nodes, such as sensors, generate manufacturing data. Edge nodes first process the data and are subjected to the cloud's in-depth analysis and decision-making support. In this system, the service selection is based on QoS when multiple services offer the same functionality. The service composition process includes workflow decomposition, service discovery, matching, and optimized selection. We focus mainly on the optimized selection, as services in the IIoT environment may face performance issues. When a service does not meet performance expectations, we need to quickly find and combine the best service from the library to maintain service continuity and quality.

B. IoT Service Dynamic Composition Model Based on Markov Decision Process

In the IIoT environment, a task typically involves multiple relatively independent sub-tasks. The execution effect of each sub-task is Markovian, unaffected by the QoS results obtained from the selection and execution of services of previous sub-tasks. Against the backdrop of industrial software service composition, a task T is usually decomposed into several sub-tasks $T = t_1, t_2, \dots, t_n$, where n is the number of sub-tasks, and t_i is the i_{th} sub-task. We model the entire service composition process as a Markov Decision Process (MDP).

An IIoT service dynamic composition model based on the Markov decision process named *ISC-MDP*, is a quintuple: $ISC-MDP = S, s_0, s_t, A, R$, where: S represents the state set, with each state $s \in S$ corresponding to service selection and composition for a particular sub-task. $s_0 \in S$ denotes the initial state, where the service composition workflow begins its execution. $s_t \in S$ signifies the termination state. Upon reaching

the termination state, the service composition process concludes. A symbolizes the action set, which is a finite set of services. Each action $a \in A$ corresponds to selecting a service for a sub-task under state s . R is the immediate reward function. If the current state is S and action a has been executed,

the immediate reward $R(s'|s, a)$ is obtained. In evaluating the performance of service compositions, we consider four key QoS attributes: reliability, availability, response time, and cost. To make these attributes, which have different measurement standards and units, comparable, we normalize them using the following formulas:

$$q_{ij}^+ = \begin{cases} \frac{q_{ij} - \min(q_{ij})}{\max(q_{ij}) - \min(q_{ij})}, & \text{if } \max(q_{ij}) \neq \min(q_{ij}) \\ 1, & \text{if } \max(q_{ij}) = \min(q_{ij}) \end{cases} \quad (1)$$

$$q_{ij}^- = \begin{cases} \frac{\max(q_{ij}) - q_{ij}}{\max(q_{ij}) - \min(q_{ij})}, & \text{if } \max(q_{ij}) \neq \min(q_{ij}) \\ 1, & \text{if } \max(q_{ij}) = \min(q_{ij}) \end{cases} \quad (2)$$

Where q_{ij}^+ and q_{ij}^- respectively indicate the j_{th} QoS positive and negative attributes of the i_{th} service, $\max(q_{ij})$ and $\min(q_{ij})$ respectively indicate the maximum and minimum values of the j_{th} QoS attribute of the i_{th} service:

Next, we further weigh the normalized QoS attributes to obtain the QoS-weighted value of the service as an immediate reward:

$$R = \sum_j \omega_j \cdot q_{ij} \quad (3)$$

Where ω_j is the weight of the j_{th} QoS attribute.

The entire service composition process can be viewed as starting from the initial state and, by selecting the appropriate services, transitioning to a new state until the terminal state is reached. Once the service composition workflow is completed, it will comprise several specific services arranged in a specific execution order to form a complete service composition.

IV. PROPOSED ALGORITHM FRAMEWORK

This research introduces the service composition approach PDQ3ND based on deep reinforcement learning. The PDQ3ND approach incorporates improvements, including double-objective strategies, adversarial structural strategies, prioritized experience replay, noisy networks, and demonstration learning. The architectural diagram of this algorithm is shown in Fig. 2.

A. Deep Q-Network

The Deep Q-Network (DQN) is a reinforcement learning method that combines Deep Neural Networks (DNN) and Q-learning [8]. By integrating a deep neural network as a function

approximator, it approximates the optimal action-value function, which satisfies the Bellman optimality equation:

$$Q^*(s, a) = E \left[r + \gamma \max_{a'} Q^*(s', a') | s, a \right] \quad (4)$$

During the learning process, DQN employs Q-learning updates, updating from uniformly randomly sampled experience replay samples from the replay buffer, and minimizing the mean squared error loss between the predicted Q-value and the target Q-value of the Bellman optimality equation:

$$L(\theta) = E \left[\left(Q(s, a; \theta) - \left(r + \gamma \max_{a'} Q(s', a'; \theta^-) \right) \right)^2 \right] \quad (5)$$

B. Double Objective Network

The double objective network primarily addresses the overestimation issue in DQN [9]. In DQN, the same network is used to select actions and calculate target values, which may lead to overestimating action values. Specifically, the double objective strategy uses the Q-network to select actions and use the target network to compute target values:

$$y = r + \gamma \hat{Q} \left(s', \arg \max_a Q(s', a; \theta); \theta^- \right) \quad (6)$$

C. Dueling Architecture Network

The dueling architecture network aims to enhance the neural network architecture, dividing the Q-network into two independent sub-networks: one responsible for learning the state value function while the other for learning the action advantage function [10]. The outputs of the two sub-networks are combined via an aggregation layer to form the overall network output:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_a A(s, a; \theta) \right) \quad (7)$$

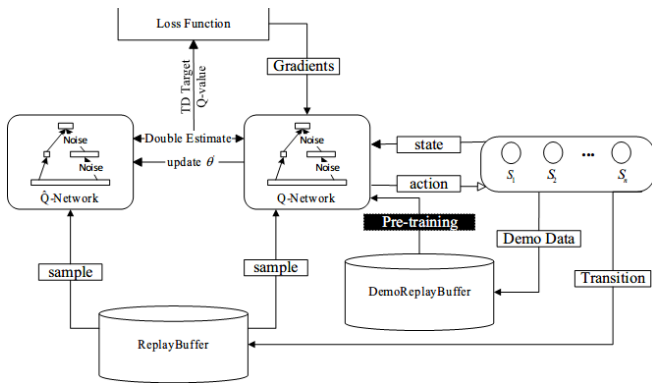


Figure 2. The Architecture of PD3QND.

D. Prioritized Experience Replay

In DQN, experience replay samples uniformly at random from the experience buffer. However, this method might

undervalue samples with essential information while oversampling less critical samples. The prioritized experience replay assigns priority based on each sample's TD error (Temporal Difference Error) during training [11]. The TD error is given by:

$$\delta = r + \gamma \hat{Q} \left(s', \arg \max_{a'} Q(s', a'; \theta); \theta^- \right) - Q(s, a; \theta) \quad (8)$$

E. Noisy Network Strategy

In DQN, the ϵ -greedy strategy is typically employed to choose action. However, this method's exploration extent is fixed and might not adaptively adjust according to different problems and training phases. The noisy network introduces Gaussian noise into the neural network's linear layer so that the network's output becomes stochastic, facilitating a balance between exploration and exploitation during training [12].

F. Expert Demonstration

Expert demonstration combines supervised learning, utilizing prior knowledge and expert demonstrations to guide the neural network training [13], accelerating the learning process and enhancing algorithm performance. The loss function comprises four parts: 1-step return, n-step return, supervision loss, and L2 regularization term. The specific loss function formula is:

$$L(\theta) = \lambda_1 * L_1(\theta) + \lambda_2 * L_n(\theta) + \lambda_3 * L_{L2}(\theta) + \lambda_4 * L_s(\theta) \quad (9)$$

Where $L_1(\theta)$ is the same as the loss function of DQN. $L_n(\theta)$ is the n-step return loss, measuring the current strategy's accuracy in predicting the cumulative reward for the next n steps. $L_{L2}(\theta)$ is the L2 regularization term loss, used to constrain the complexity of the neural network and prevent overfitting. $L_s(\theta)$ is the supervision loss, measuring the difference between the agent's current and the expert strategies. Here, we use the action value of the expert strategy as the target for supervised learning, guiding the agent to prefer actions whose Q-values $Q^*(s, a_E)$ are closer to the demonstration action Q-value. The formula is:

$$L_s = \max_a [Q(s, a) - l(a_E, a)] - Q^*(s, a_E) \quad (10)$$

V. ALGORITHM DESIGN AND IMPLEMENTATION

To verify the performance of the proposed algorithm, we conducted experiments based on the QWS dataset. The purpose of the experiment was to compare the performance of GA, DQN, and PD3QND in terms of scalability and adaptability. Expert data for PD3QND are sourced from the top three rewards out of 5,000 random compositions.

A. Efficiency Comparison

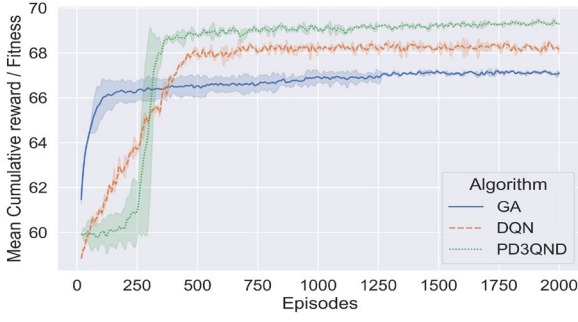


Figure 3. Training/iteration curves for PD3QND, DQN, and GA.

During the training/iteration process of 2000 episodes, the curve of PD3QND was noticeably higher than the other two algorithms (Fig. 3). Notably, PD3QND effectively addressed the cold start problem, allowing it to converge to quality solutions in the initial iterations quickly. In contrast, DQN performed similarly to PD3QND in the initial episodes but slowed down in subsequent episodes.

B. Scalability Comparison

We simulated the performance of various algorithms under different numbers of sub-tasks. Mainly when the number of sub-tasks is 90 (Fig. 4), the advantage of PD3QND is more evident. As the complexity of the tasks increases, the PD3QND algorithm can adapt better and achieve higher performance.

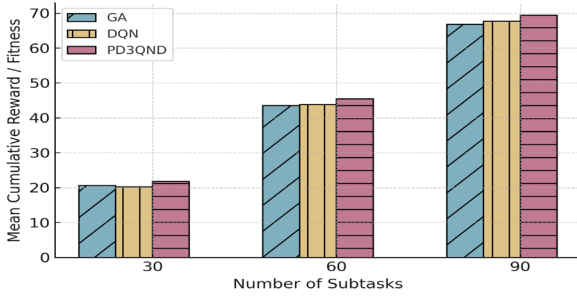


Figure 4. Reward/fitness for a different number of subtasks.

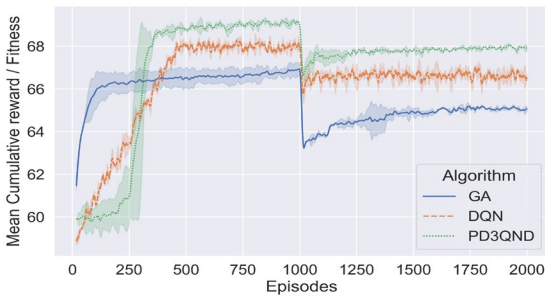


Figure 5. Adaptability for PD3QND, DQN, and GA.

C. Adaptability Comparison

To evaluate the adaptability of each algorithm, we simulated a scenario where 10% of the services became unavailable at the 1000th episode. In the meantime, both DQN

and GA lagged behind PD3QND in terms of recovery and growth speed (Fig. 5). This indicates that PD3QND can swiftly adapt and maintain efficient performance even faced with unexpected situations like service unavailability.

VI. CONCLUSION

In this study, we proposed a service composition approach, PD3QND, based on deep reinforcement learning, tailored for the service composition challenges in the Internet of Things environment. Compared with GA and DQN algorithms, PD3QND better adapts to large-scale dynamic service composition challenges and strikes an effective balance between exploration and exploitation. Moreover, it effectively addresses the cold start problem, allowing it to quickly converge to high-quality solutions during initial iterations, significantly enhancing the algorithm's efficiency.

ACKNOWLEDGMENT

This work is supported by the National Natural Science Foundation of China under Grant (U20A6003).

REFERENCES

- [1] Y. Yang, B. Yang, S. Wang, F. Liu, Y. Wang, and X. Shu, "A dynamic ant-colony genetic algorithm for cloud service composition optimization," *Int. J. Adv. Manuf. Technol.*, vol. 102, pp. 355–368, 2019.
- [2] Z. Zhang, X. Guo, M. Zhou, S. Liu, and L. Qi, "Multi-objective discrete grey wolf optimizer for solving stochastic multi-objective disassembly sequencing and line balancing problem," in *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, IEEE, 2020, pp. 682–687.
- [3] V. Rajendran, R. K. Ramasamy, and W.-N. Mohd-Isa, "Improved eagle strategy algorithm for dynamic web service composition in the IoT: a conceptual approach," *Future Internet*, vol. 14, no. 2, p. 56, 2022.
- [4] X. Ren et al., "DeepQSC: A GNN and attention mechanism-based framework for QoS-aware service composition," in *2021 International Conference on Service Science (ICSS)*, IEEE, 2021, pp. 76–83.
- [5] H. Wang et al., "Integrating reinforcement learning and skyline computing for adaptive service composition," *Inf. Sci.*, vol. 519, pp. 141–160, 2020.
- [6] K. Yi, J. Yang, S. Wang, Z. Zhang, and X. Ren, "PPDRL: A Pretraining-and-Policy-Based Deep Reinforcement Learning Approach for QoS-Aware Service Composition," *Secur. Commun. Netw.*, vol. 2022, 2022.
- [7] Y. Liu et al., "Logistics-involved service composition in a dynamic cloud manufacturing environment: A DDPG-based approach," *Robot. Comput.-Integr. Manuf.*, vol. 76, p. 102323, 2022.
- [8] V. Mnih et al., "Playing atari with deep reinforcement learning," *ArXiv Prepr. ArXiv13125602*, 2013.
- [9] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the AAAI conference on artificial intelligence*, 2016.
- [10] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *International conference on machine learning*, PMLR, 2016, pp. 1995–2003.
- [11] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *ArXiv Prepr. ArXiv151105952*, 2015.
- [12] M. Fortunato et al., "Noisy networks for exploration," *ArXiv Prepr. ArXiv170610295*, 2017.
- [13] T. Hester et al., "Deep q-learning from demonstrations," in *Proceedings of the AAAI conference on artificial intelligence*, 2018.